



**PRATHYUSHA
ENGINEERING COLLEGE**

An **Autonomous** Institution

SMART IRRIGATION SYSTEM

Industrial Internship Report on
ESP32-Based Embedded Systems and IoT Project

Prepared as a Project Report
Academic / Internship Project

Prepared by

[Student Name]

Prathyusha Engineering College

An Autonomous Institution

Final Project – Free Summer Internship in Embedded and IoT – 2026
upskill Campus | The IoT Academy | UniConverge Technologies Pvt Ltd (UCT)

Date of Submission: 30 August 2026

Wokwi Project: <https://wokwi.com/projects/473414532081154049>

Executive Summary

This report presents the Smart Irrigation System developed as the final project for the Free Summer Internship in Embedded and IoT – 2026. The project follows the structure and submission expectations shown in the Upskill Campus / UCT sample report supplied for this internship. The sample report asks the student to document the problem, existing and proposed solution, design, interfaces, performance testing, learnings, future scope and GitHub submission links. filecite turn2file2

The project uses an ESP32 development board, an SSD1306 128×64 OLED display, a relay module and a potentiometer-based input in the Wokwi simulation. The program maintains a simulated soil-moisture value, compares it with a threshold of 2000 and changes the relay state accordingly. When the value is below 2000, the program treats the soil as dry and turns the relay ON; at 2000 or above, it treats the soil as wet and turns the relay OFF.

A key implementation detail is that the supplied program currently simulates the soil value in software rather than reading the potentiometer on GPIO 34 with an analog input function. The Wokwi circuit nevertheless includes the potentiometer and connects its signal to GPIO 34. This report therefore describes the present implementation accurately and identifies physical sensor integration as future work.

TABLE OF CONTENTS

- 1 Preface
- 2 Introduction
 - 2.1 About UniConverge Technologies Pvt Ltd
 - 2.2 About upskill Campus (USC)
 - 2.3 The IoT Academy
 - 2.4 Objectives of this Internship Program
 - 2.5 Reference
 - 2.6 Glossary
- 3 Problem Statement
- 4 Existing and Proposed Solution
 - 4.1 Code Submission (GitHub Link)
 - 4.2 Report Submission (GitHub Link)
- 5 Proposed Design / Model
 - 5.1 High Level Diagram
 - 5.2 Low Level Diagram
 - 5.3 Interfaces
- 6 Performance Test
 - 6.1 Test Plan / Test Cases
 - 6.2 Test Procedure
 - 6.3 Performance Outcome
- 7 My Learnings
- 8 Future Work Scope
- Conclusion
- Appendix – Source Code

1 Preface

The six-week internship provided an opportunity to connect classroom learning with practical embedded and IoT work. The supplied Upskill Campus sample describes the internship as an opportunity to gain practical industry experience, solve real-world problems, improve job prospects, understand the field and its applications, and develop communication and problem-solving skills. filecite turn2file1

During this project, I worked on a Smart Irrigation System using an ESP32 and Wokwi simulation. The main idea was to automate irrigation decisions using a soil-condition value, display the status on an OLED and control an irrigation output through a relay.

The project work involved understanding the circuit, preparing the ESP32 program, interfacing the OLED through I2C, implementing threshold logic, observing serial output, testing dry and wet conditions and documenting the result. The Wokwi environment made it possible to repeat the tests safely without using a physical pump.

This internship also helped me understand the importance of designing for practical constraints. Because the current project is a simulation and the soil value is generated in software, real sensor accuracy, pump performance, power consumption and field durability could not be measured. These limitations are documented instead of being presented as completed hardware tests.

I am thankful to the internship organizers, mentors and faculty members who supported the learning process. I also see this project as a useful foundation for a future version with a real soil-moisture sensor, Wi-Fi monitoring and a safe physical pump-control circuit.

Message to juniors and peers: Start with a small working system, test every block separately and document what you actually implemented. Clear documentation is as important as the code when

presenting an engineering project.

2 Introduction

Smart irrigation is an application area where embedded systems can be used to make watering decisions more responsive to soil conditions. A simple controller can monitor a condition, compare it with a predefined limit and operate an actuator only when required. The project demonstrates this basic control loop using ESP32, a relay and an OLED display.

The internship sample provided for this report places the project in the context of industrial exposure and asks students to describe the internship opportunity, design approach, performance testing and learning outcomes. filecite turn2file2

2.1 About UniConverge Technologies Pvt Ltd

The supplied sample report states that UniConverge Technologies Pvt Ltd (UCT) was established in 2013 and works in the Digital Transformation domain, with a focus on sustainability and return on investment. It lists technologies including IoT, cyber security, cloud computing, machine learning, 4G/5G/LoRaWAN communication, Java Full Stack, Python and front-end technologies. filecite turn2file1

The sample further describes UCT Insight as an IoT platform supporting device connectivity through MQTT, CoAP, HTTP, Modbus TCP and OPC UA, with cloud and on-premises deployment options. It also mentions dashboard building, analytics and reporting, alerts and notifications, third-party integrations and a rule engine. filecite turn2file1

The sample describes Factory Watch as a smart-factory platform for production and asset monitoring, OEE, predictive maintenance and scaling toward digital-twin solutions. It also states that UCT has adopted LoRaWAN for solutions in Agritech, smart cities, industrial monitoring and smart metering, and uses embedded systems, Industrial IoT and machine learning for predictive maintenance. filecite turn2file0

2.2 About upskill Campus (USC)

According to the supplied sample, upskill Campus, together with The IoT Academy and in association with UniConverge Technologies, facilitated the execution of the complete internship process. The sample describes USC as a career-development platform. filecite turn2file1

2.3 The IoT Academy

The supplied sample describes The IoT Academy as the EdTech division of UCT and states that it runs long executive certification programs in collaboration with EICT Academy, IIT Kanpur, IIT Roorkee and IIT Guwahati in multiple domains. filecite turn2file1

2.4 Objectives of this Internship Program

The internship objectives stated in the supplied sample are to gain practical industry experience, solve real-world problems, improve job prospects, improve understanding of the field and its applications, and develop personal skills such as communication and problem solving. filecite turn2file2

For this project, those objectives were addressed through embedded-system design, Wokwi simulation, ESP32 programming, interface configuration, threshold-based control, testing and technical documentation.

2.5 Reference

[1] Upskill Campus / UCT Sample Internship Report supplied for the internship.

[2] Wokwi project: <https://wokwi.com/projects/473414532081154049>

[3] Arduino Documentation: <https://docs.arduino.cc/>

[4] Espressif ESP32 Documentation: <https://docs.espressif.com/>

[5] Adafruit SSD1306 and GFX documentation: <https://learn.adafruit.com/>

2.6 Glossary

Term	Meaning
ESP32	Microcontroller development platform used as the main controller.
IoT	Internet of Things; connected sensing, processing and control systems.
OLED	Organic Light-Emitting Diode display used to show system status.
I2C	Two-wire serial communication interface used by the OLED.
GPIO	General Purpose Input/Output pin on the ESP32.
Relay	Electrically controlled switch used here as the pump-control output.
Threshold	Preset value used to decide whether irrigation should be ON or OFF.
Wokwi	Online electronics simulator used to build and test the prototype.

3 Problem Statement

Manual irrigation depends on a person observing plants and deciding when to water them. Timer-based irrigation can also water plants at fixed times even when the soil condition does not require it. The project addresses this basic problem by demonstrating a controller that makes an irrigation decision from a soil-condition value.

The target behavior is simple: when the soil is considered dry, the irrigation output should turn ON; when the soil is considered wet, the output should turn OFF. The system should also provide a visible status so that the user can understand the current value and pump state.

Current project scope: The Wokwi circuit contains a potentiometer connected to ESP32 GPIO 34 as the simulated soil input, but the supplied program does not currently read GPIO 34. Instead, it automatically changes the variable **soilValue** between wet and dry ranges. Therefore, the present submission demonstrates the control logic and simulation rather than a calibrated physical soil-moisture measurement.

4 Existing and Proposed Solution

Existing approaches: Commonsimpleirrigationapproaches include manual watering and timer-based watering. Manual watering requires regular attention, while a fixed timer does not directly respond to the current soil condition.

Proposed solution: The ESP32 runs a threshold-based irrigation controller. The program starts with a soil value of 3000, decreases it by 100 while the simulated soil is drying, and then increases it by 100 while the simulated soil is wetting. The threshold is 2000.

Control rule: If **soilValue < 2000**, GPIO 25 is set LOW and the relay is treated as ON. If **soilValue ≥ 2000**, GPIO 25 is set HIGH and the relay is treated as OFF.

Value addition: The prototype combines sensing logic, decision making, actuation and user feedback in one embedded system. It is repeatable in Wokwi and provides a clear base for later integration with a real sensor, pump and IoT monitoring service.

4.1 Code Submission (GitHub Link)

Repository name: upskillcampus

Code filename: SmartIrrigationSystem.ino

Code link: [<https://github.com/prudhvikancharla3/upskillcampus/blob/main/SmartIrrigationSystem.ino>]

The supplied Upskill Campus instructions require the repository to be named upskillcampus and require the project code file to be uploaded in the repository. filecite turn2file3

4.2 Report Submission (GitHub Link)

Report filename: SmartIrrigationSystem_[YourName]_USC_UCT.pdf

Report link: [PASTE YOUR CURRENT GITHUB REPORT LINK HERE]

The supplied instructions say to first create the report-link placeholder, copy the working GitHub link, update the report with the current working links, and upload the updated report to GitHub to replace the old report. filecite turn2file3

5 Proposed Design / Model

The system is organized as a simple embedded control loop. A soil-condition value is provided to the ESP32. The controller compares the value with the threshold and drives the relay output. The OLED provides local feedback and the Serial Monitor provides a debugging view.

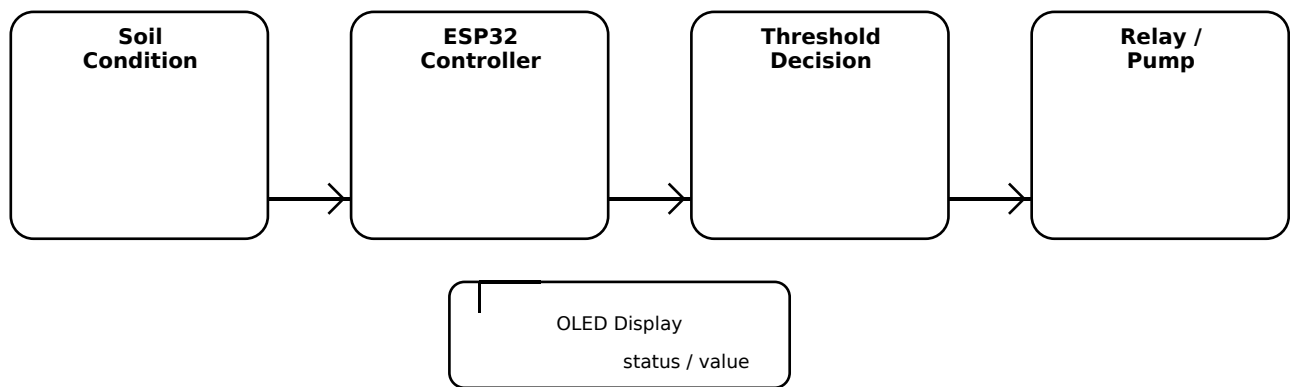


Figure 1: High-level functional flow of the Smart Irrigation System.

5.1 High Level Diagram

The high-level flow is: soil condition → ESP32 processing → threshold decision → relay/pump control. The OLED is used to present the current value and decision, while Serial Monitor output is used during testing.

5.2 Low Level Diagram

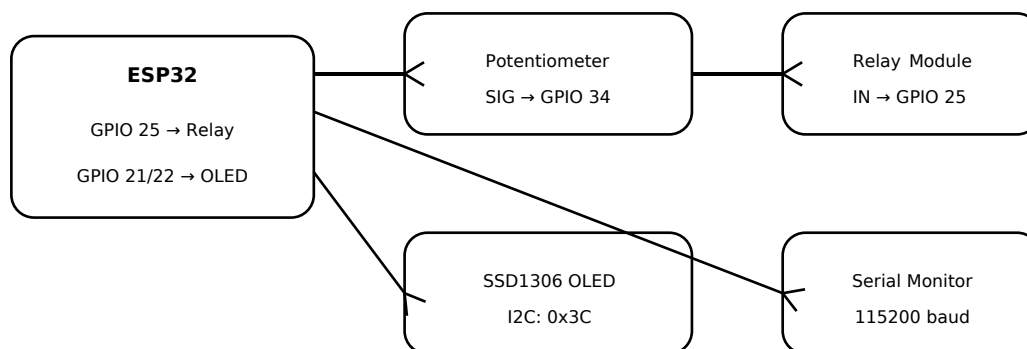


Figure 2: Low-level interfaces derived from the Wokwi circuit and supplied source code.

The Wokwi diagram connects the potentiometer signal to GPIO34, the relay input to GPIO 25, OLED SDA to GPIO 21 and OLED SCL to GPIO 22. The OLED uses I2C address 0x3C. The source code defines GPIO 25 as the relay-control pin and initializes the OLED at 0x3C.

5.3 Interfaces

Interface	Connection / Setting	Purpose
Soil input	Potentiometer SIG → GPIO 34	Represents soil-condition input in the Wokwi circuit.
Relay control	Relay IN → GPIO 25	Controls the irrigation output state.
OLED SDA	GPIO 21	I2C data line.
OLED SCL	GPIO 22	I2C clock line.
OLED address	0x3C	SSD1306 I2C device address.
Serial	115200 baud	Displays the soil value for monitoring/debugging.

6 Performance Test

The supplied Upskill Campus sample emphasizes performance testing as an important part of demonstrating why the work is relevant to industry rather than being only an academic exercise. It asks the student to identify constraints, explain how they were handled and report the test results; if a constraint could not be tested, the report should explain its impact and recommend how it can be handled. filecite turn3file3

For this simulation, the main constraints considered were decision correctness, update interval, display behavior, repeatability and the limits of simulation. The code updates the simulated value once per second because the loop ends with a one-second delay. The threshold logic is deterministic, so the same value should always produce the same relay decision.

Important limitation: Because the current implementation is simulated, this report does not claim real pump flow rate, soil-sensor accuracy, electrical power consumption, actuator durability or field irrigation performance. These require physical hardware testing.

6.1 Test Plan / Test Cases

Test	Input / Condition	Expected Result	Observed Logic
TC01	soilValue = 3000	Wet; Pump OFF	Pass – value is ≥ 2000 .
TC02	soilValue = 2000	Wet; Pump OFF	Pass – equality remains in wet state.
TC03	soilValue = 1900	Dry; Pump ON	Pass – value is < 2000 .
TC04	soilValue < 1000	Dry; Pump ON; direction changes later	Pass – drying direction changes below 1000.
TC05	soilValue rises above 2000	Wet; Pump OFF	Pass – wet state is restored.
TC06	Repeated drying/wetting cycle	Consistent switching	Pass – automatic cycle repeats by program logic.

6.2 Test Procedure

1. Open the Wokwi project and start the simulation.
2. Observe the startup message on the OLED.
3. Allow the program to reduce the simulated soil value.
4. Verify that the relay state changes to ON when the value becomes less than 2000.
5. Observe the OLED messages and Serial Monitor output.
6. Allow the value to increase again and verify that the relay returns to OFF at 2000 or above.
7. Repeat the cycle and confirm consistent behavior.

6.3 Performance Outcome

The threshold-based control behavior is deterministic and repeatable in simulation. The OLED status follows the same decision used for the relay output. The automatic drying/wetting sequence makes it possible to test both states without manually changing a sensor.

The one-second loop delay provides a simple periodic update. For a real irrigation system, the update period would need to be selected based on the sensor response, plant needs and control requirements. Physical safety and actuator constraints would also need to be tested before deployment.

The project therefore meets its present simulation objective, while real-world measurements remain future validation work.

7 My Learnings

The project strengthened my understanding of ESP32 programming and the structure of an embedded application. I learned how setup and loop functions are organized, how GPIO outputs are controlled and how threshold logic can be used to make an automatic decision.

I also learned about OLED display interfacing through I2C, including the use of the Wire library and SSD1306 display library. Using Serial Monitor output helped me understand how internal values can be observed while debugging.

Working with Wokwi improved my ability to build and inspect a circuit before moving to physical hardware. The project also showed me why the software implementation and circuit diagram must be checked together: the current diagram includes a potentiometer on GPIO 34, while the supplied code simulates the value instead of reading that pin.

From an internship perspective, I learned the importance of testing edge cases, recording limitations honestly and presenting a project in a structured way. These skills will be useful when moving from academic prototypes toward practical embedded and IoT applications.

8 Future Work Scope

The supplied sample allows ideas that could not be completed because of time limitations to be listed as future work. filecite turn3file3

1. Replace the software-generated soil value with a calibrated physical soil-moisture sensor and read it through ESP32 ADC input.
2. Add Wi-Fi connectivity for remote monitoring and control.
3. Send moisture and pump data to an IoT/cloud dashboard.
4. Add adjustable moisture thresholds instead of a fixed value of 2000.
5. Add rain detection and water-tank level sensing.
6. Add pump dry-run protection and flow monitoring.
7. Add alerts for abnormal moisture, empty tank or pump faults.
8. Support multiple irrigation zones.
9. Measure power consumption and validate relay/pump durability.
10. Build a safe physical prototype with appropriate isolation, power supply and protection.

Conclusion

The Smart Irrigation System demonstrates a complete embedded control concept using ESP32, threshold-based decision making, relay control and OLED feedback. The Wokwi simulation provides a repeatable environment for observing dry and wet operating states.

The project successfully covers the main stages expected in the supplied Upskill Campus report structure: problem definition, proposed solution, design, interfaces, performance testing, learning outcomes and future scope. filecite turn2file2

The most important limitation is that the present program simulates the soil value instead of reading the potentiometer/physical sensor. The next development step is therefore to connect the actual input to the control algorithm and then validate the complete system with a real pump and safe power-control hardware.

Appendix - Project Source Code

The following is the source code contained in the submitted Wokwi project ZIP. The code uses the Adafruit GFX and Adafruit SSD1306 libraries listed in the project's libraries.txt file.

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64

#define OLED_RESET -1
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);

#define RELAY_PIN 25
#define THRESHOLD 2000

int soilValue = 3000; // Start as wet
```

```

bool drying = true;    // Direction of change

void setup() {
  Serial.begin(115200);
  pinMode(RELAY_PIN, OUTPUT);

  if(!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    Serial.println(F("SSD1306 allocation failed"));
    for(;;);
  }

  display.clearDisplay();
  display.setTextSize(1);
  display.setTextColor(SSD1306_WHITE);
  display.setCursor(0, 0);
  display.println("Hello Rancher!");
  display.println("");
  display.println("Monitoring soil moisture...");
  display.display();
  delay(3000);
}

void loop() {
  // Simulate soil drying/wetting automatically
  if (drying) {

    soilValue -= 100; // soil dries
    if (soilValue < 1000) drying = false; // switch to wetting
  } else {
    soilValue += 100; // soil gets wet
    if (soilValue > 3000) drying = true; // switch to drying
  }
  Serial.println(soilValue);

  display.clearDisplay();
  display.setCursor(0, 0);
  display.setTextSize(1);
  display.setTextColor(SSD1306_WHITE);

  display.println("Soil Value: " + String(soilValue));
  display.println("");

  if (soilValue < THRESHOLD) {
    digitalWrite(RELAY_PIN, LOW); // Relay ON

    display.println(" Soil is Dry");
    display.println("");
    display.println("Pump: ON ");
    display.println("");
    display.println("Pumping water...");
  } else {
    digitalWrite(RELAY_PIN, HIGH); // Relay OFF
    display.println(" Soil is Wet");
    display.println("");
    display.println("Pump: OFF ");
    display.println("");
    display.println("Saving water...");
  }

  display.display();
  delay(1000);
}

```

Required libraries

Wokwi Library List
 # See <https://docs.wokwi.com/guides/libraries>

Automatically added based on includes:

Adafruit GFX Library
 Adafruit SSD1306

Project Circuit

